

MERN Backend Concepts Cheat Sheet – Ayush Verma

1■■■ Middleware in Express:

- Middleware are functions that run between request and response.
- Used for authentication, logging, validation, and error handling.
- Example: `fetchUser` middleware verifies JWT token before route access.

2■■■ How JWT Works Internally:

- JWT = Header.Payload.Signature
- Header defines algorithm, Payload contains user data, Signature verifies authenticity.
- Created using `jwt.sign()`, verified using `jwt.verify()`.
- Stateless authentication – no session storage needed.

3■■■ Role of ObjectId in MongoDB:

- Each MongoDB document has a unique ObjectId (`_id`).
- 12-byte structure (timestamp + random + counter).
- Used to identify or reference other documents (relations).

4■■■ Relational Data in MongoDB:

- MongoDB is NoSQL but supports relationships using:
 - References (ObjectId + ref)
 - Embedded subdocuments
- Example: Each Note stores user's ObjectId to link Note ↔ User.

5■■■ CRUD Operations (Quick Recap):

- Create – POST `/addnote` → `note.save()`
- Read – GET `/fetchallnotes` → `Note.find()`
- Update – PUT `/updatenote/:id` → `findByIdAndUpdate()`
- Delete – DELETE `/deletenote/:id` → `findByIdAndDelete()`

6■■■ Async/Await Importance:

- Ensures database operations finish before sending response.
- Without it, code runs asynchronously and may send incomplete data.
- `async/await` replaces messy `.then()` chains and improves readability.

7■■■ CORS & `express.json()`:

- `cors()` → allows frontend (port 3000) to access backend (port 5000).
- `express.json()` → parses incoming JSON request bodies.

■ Tip: Use `try/catch` around all async routes to handle runtime errors safely.